PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Proyecto Express

Presentado a: **Yoly Alexandra Guevara Muñoz**

Por Aprendiz: **Karen Vanessa Castañeda Moran**

Ficha: **3312932**

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día **20** de **octubre** del año **2026**

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

INTRODUCCION

Este documento detalla la construcción de una App desarrollada con Node.js y Express para el programa ADSO del SENA. El objetivo central es implementar la arquitectura backend fundamental mediante el diseño de rutas (GET y POST), la configuración de middlewares para el procesamiento de datos JSON y el registro de logs, junto con la ejecución de pruebas funcionales que validan el correcto comportamiento del servidor.

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

1. **¿Qué es un middleware en Express y en qué momento del ciclo de una petición se ejecuta?**

R//

Es una función intermedia que tiene acceso a los objetos de solicitud (req), de respuesta (res) y a la siguiente función de middleware en el ciclo de solicitud-respuesta de la aplicación. Se ejecuta después de que el servidor recibe la petición del cliente y antes de que la ruta final envíe una respuesta. Sirve para transformar datos, ejecutar validaciones de seguridad o interceptar peticiones.

2. **¿Qué significan los objetos req y res?**

R//

- **req (Request): Representa la petición realizada por el cliente.**
- **res (Response): Representa la respuesta que el servidor envía de vuelta al cliente.**

3. **¿Cuál es la diferencia entre un middleware global, uno aplicado a una ruta específica y uno de manejo de errores?**

R//

- **Global: Se registra mediante las rutas.**
- **De ruta específica: Se inyecta directamente como argumento en un endpoint determinado y solo se ejecuta al consultar esa URL específica.**
- **De manejo de errores: Se define obligatoriamente con cuatro parámetros. Express lo identifica de forma especial para capturar excepciones o fallos lanzados en cualquier parte del código y dar una respuesta controlada al cliente.**

4. **¿Qué hace express.json() y qué problema se presenta si no se configura cuando el cliente envía JSON?**

R//

Es un middleware integrado en Express que actúa como un *parser*. Se encarga de analizar el cuerpo de las peticiones entrantes con formato JSON y transformarlo en un objeto JavaScript legible. Si no se configura, req.body llegará como undefined, impidiendo que el servidor pueda leer o procesar los datos enviados en el cuerpo del *request*.

5. **¿Por qué importa el orden en que se registran los middlewares y las rutas?**

R//

Porque Express tiene una arquitectura secuencial que procesa las funciones en el estricto orden en que fueron declaradas en el código. Si una ruta se ubica antes de un middleware indispensable, ruta se ejecutará sin aplicar dicho filtro, rompiendo la lógica o comprometiendo la seguridad de la aplicación.

6. **¿Qué puede ocurrir si un middleware no envía una respuesta ni permite continuar el flujo?**

R//

La petición entrará en un estado de suspensión indefinido. Al no invocarse la función next() ni cerrarse el ciclo con un método del objeto res, el cliente mantendrá la conexión abierta esperando una respuesta hasta que ocurra un error por tiempo de espera.

7. **¿Cuál es la diferencia entre un middleware y la función que atiende una ruta?**

R//

- **El middleware es una pieza intermedia y reutilizable cuyo propósito es preprocesar, validar o transformar la petición de manera general o modular.**
- **La función de la ruta es el destino final del flujo de trabajo, encargada de resolver la lógica de negocio específica.**

8. **¿Qué riesgos o problemas podrían aparecer si una validación o una comprobación de acceso se omite?**

R//

- **Inyección de datos corruptos o maliciosos que podrían tumbar el servidor.**
- **Acceso no autorizado, permitiendo la fuga, alteración o eliminación de información sensible en la base de datos.**

9. **Ejemplo cotidiano sin código**

R//

El control de seguridad de un aeropuerto:

Imagina que el destino final (la ruta) es abordar el avión. El middleware global es el control de metales por el que pasan todos los pasajeros obligatoriamente. Un middleware de ruta específica es el abordaje prioritario en la puerta de embarque de primera clase. El middleware de errores es el equipo médico que interviene si un pasajero sufre un accidente durante la fila. Si pasas los controles con éxito (next()), abordas el avión; si olvidas el pasaporte, el guardia te detiene y te devuelve (res.send).

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

DESARROLLO DEL MIDDLEWARE

app.js > PORT

```
1   const express = require("express");
2   const app = express();
3   const PORT = 3000;
4   app.use(express.json());
5
6   app.get("/inicio", (req, res) => {
7       res.json({ "mensaje": "Bienvenido A Mi Plataforma" });
8   });
9
10  app.get("/informacion", (req, res) => {
11      res.json({ "mensaje": "Datos Del Servicio" });
12  })
13
14  app.post("/solicitudes", (req, res) => {
15      const { nombre, apellido, correo } = req.body;
16      if (!correo.includes('@') || !correo.includes('.')) {
17          return res.status(400).json({
18              error: "Ingrese Corre Electronico Valido"
19          });
20      }
21
22      res.json({
23          "nombre": nombre,
24          "apellido": apellido,
25          "correo": correo,
26          "mensaje": "Solicitud recibida"
27      })
28  })
29  /*puerto*/
30  app.listen(PORT, () => {
31      console.log("servidor ejecutandose correctamente " + PORT);
32  });
```

¿Dónde se ubicó el middleware propio y por qué se eligió esa ubicación?

R// Se está ubicando justo después del express.json() y justo antes de las rutas. Esta ahí para poder ejecutar cada petición que llegue antes de que el servidor procese las rutas.

¿Qué cambia si el middleware se coloca después de las rutas en vez de antes?

R// El middleware no se ejecutará. Al estar ubicado abajo, las rutas responderán primero y cerrarán el ciclo de la petición, impidiendo que el flujo de código llegue al middleware de registro.

¿Cómo se transmite el control al siguiente middleware o a la ruta?

R// Utilizando la función next(), que le avisa a Express que el middleware terminó su tarea y puede continuar con el siguiente paso.

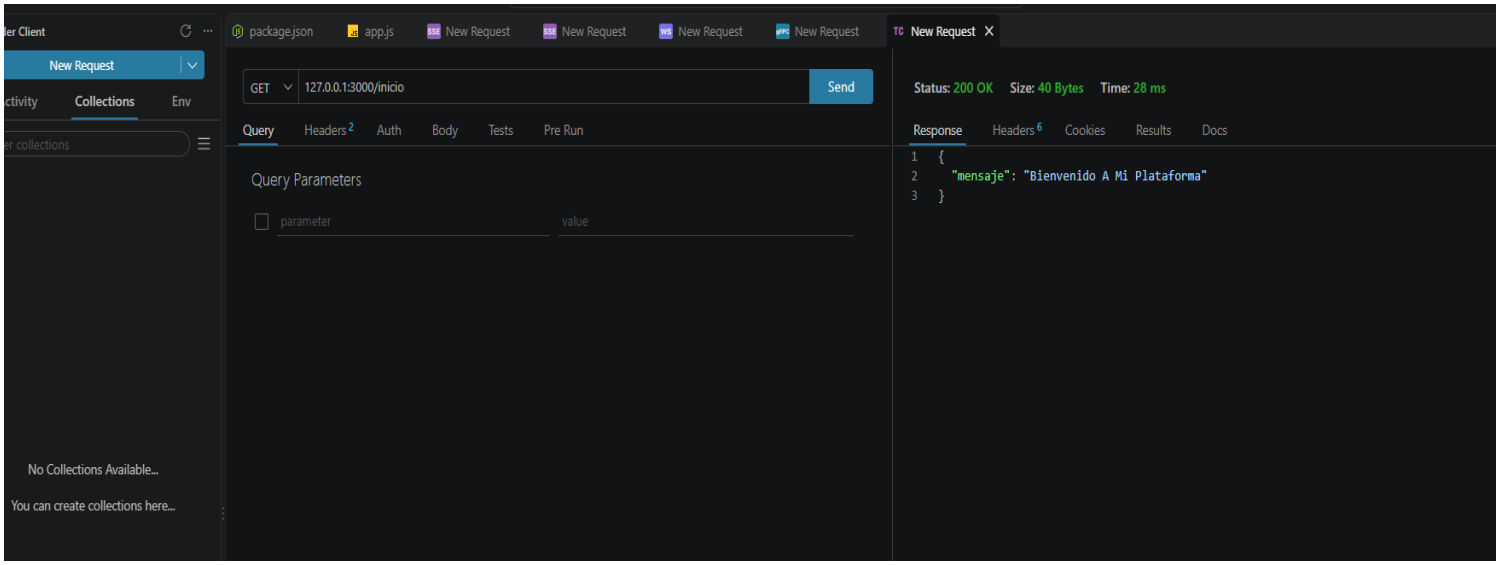
6. Códigos de estado HTTP y manejo de errores

- **200 OK: Se usa cuando una petición se procesa con éxito total, por ejemplo, al consultar las rutas GET /inicio o GET /informacion.**
- **201 Created: Indica que la petición se procesó bien y que se creó un nuevo registro o solicitud, ideal para cuando se envían datos con POST /solicitudes.**
- **400 Bad Request: Se usa cuando el cliente envía datos incompletos, erróneos o faltantes, por lo que el servidor no puede procesarlos.**
- **404 Not Found: Significa que la ruta o dirección que está buscando el cliente no existe en el servidor.**
- **500 Internal Server Error: Ocurre cuando hay un fallo inesperado del lado del servidor (un error de programación o base de datos que no se controló)**

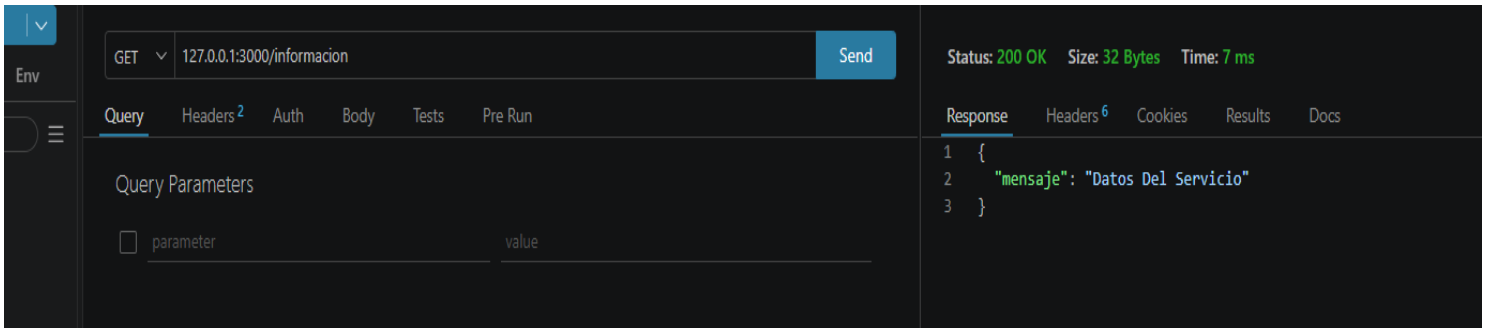
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

DESARROLLO 2

GET/INICIO: Se consulta la ruta y se verifica que devuelva la respuesta con un mensaje formato JSON, con un resultado exitoso.

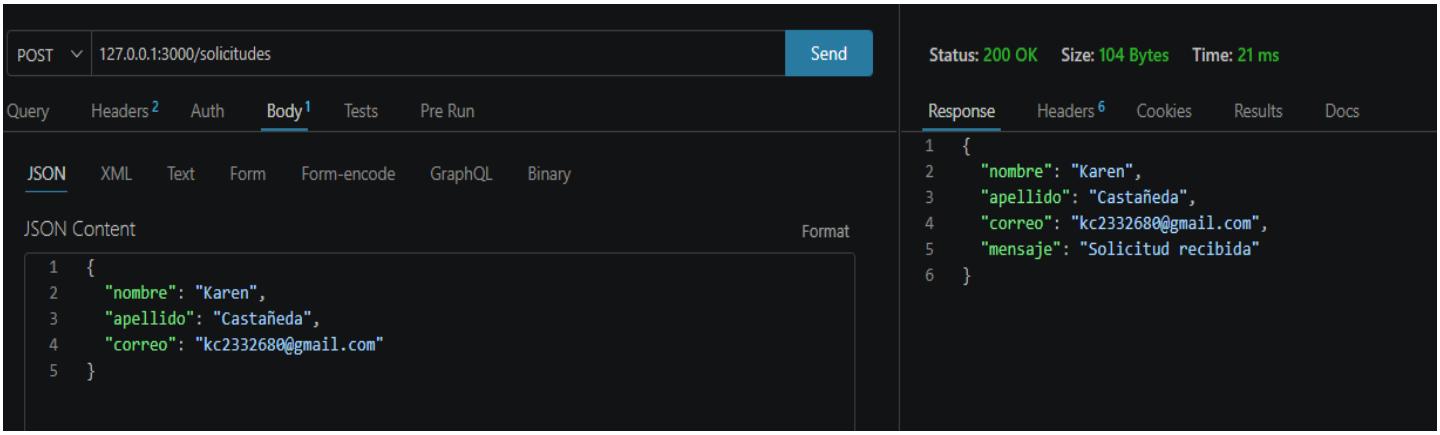


GET/INFORMACION: Se podrá comprobar que al ingreso de esta ruta el servidor responderá de manera correcta con la información del servidor.

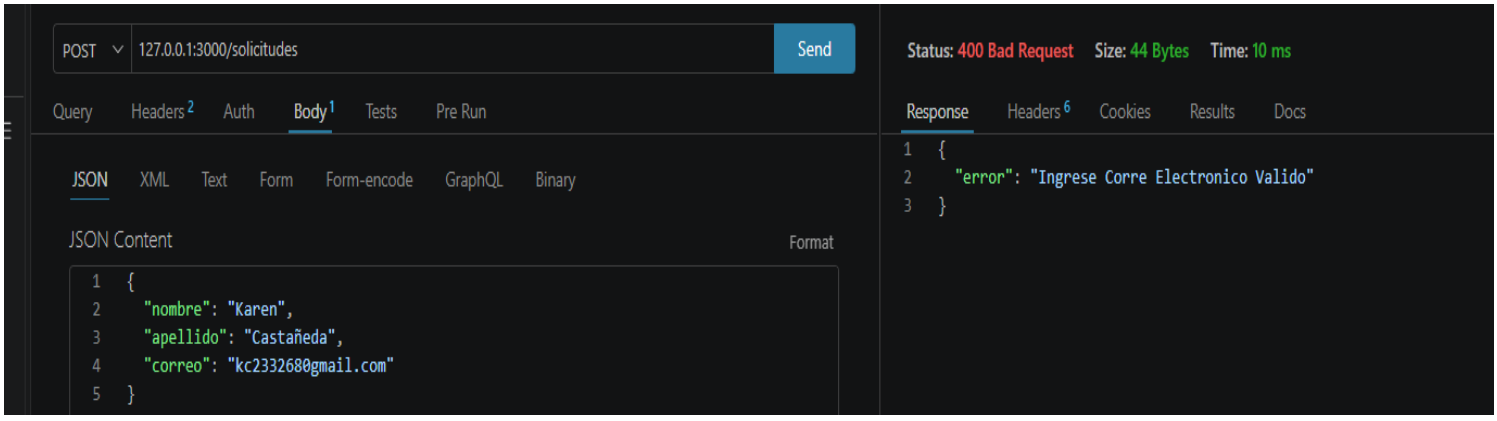


PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

POST/SOLICITUDES: Envía un cuerpo en formato JSON con nombre, apellido y el correo. Verificando la app si responde un confirmado de solicitud recibida.

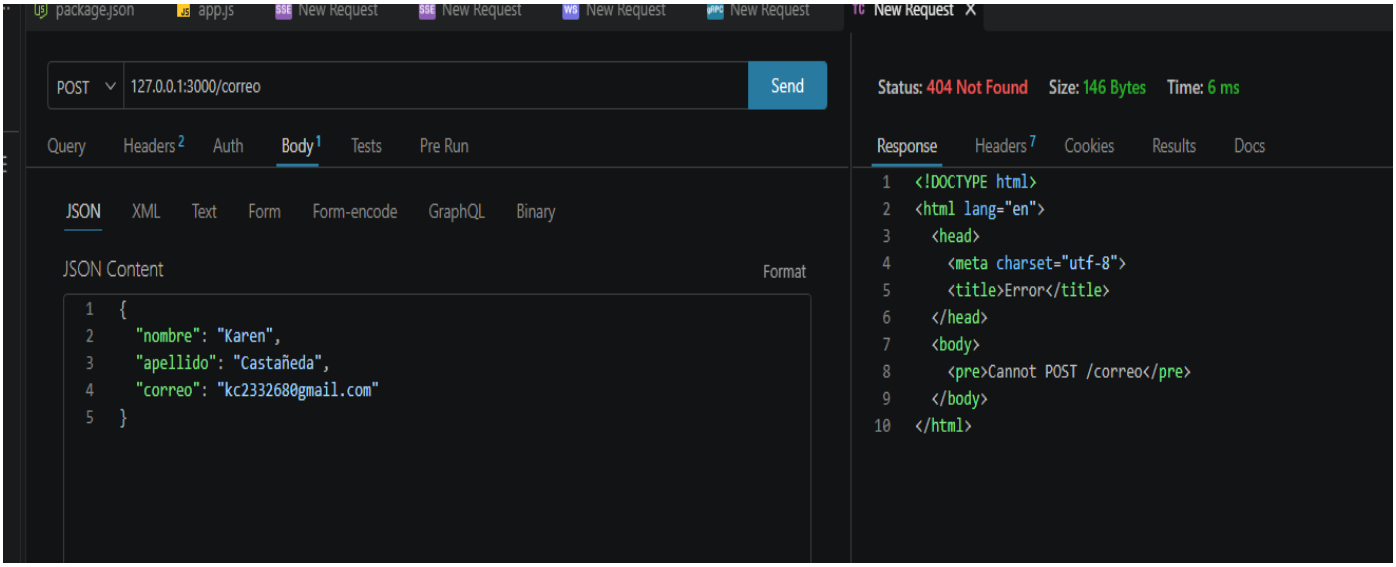


POST/SOLICITUDES: Enviara una petición de datos, modificando el correo con carácter (@). Verificando que la app este respondiendo con error.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Consultar Una Ruta Que No Exista Y Verificar La Respuesta 404:



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

1. **Instalación del entorno de ejecución (Node.js y npm)**

R//

- **Propósito: Proporcionar el entorno de ejecución para JavaScript en el lado del servidor y el gestor de paquetes del ecosistema.**

- **Impacto por omisión: Imposibilidad de inicializar el entorno, ejecutar scripts de backend o gestionar librerías de terceros.**

R//

2. **Creación del directorio y apertura del espacio de trabajo**

R//

- **Propósito: Establecer un espacio de trabajo aislado y estructurado para los componentes de la aplicación.**

- **Impacto por omisión: Desorganización de dependencias, conflictos de rutas en el sistema de archivos local y fallos en el control de versiones.**

3. **Inicialización del proyecto y generación del package.json**

R//

- **Propósito: Crear el manifiesto del proyecto que gestiona los metadatos, los scripts de ejecución y el árbol de dependencias.**
- **Impacto por omisión: Node.js no puede registrar los módulos instalados ni rastrear las dependencias necesarias para el despliegue del proyecto.**

4. **Instalación del (Express.js)**

R//

- **Propósito: Proveer una infraestructura minimalista y flexible para la gestión de enrutamiento, middlewares y solicitudes HTTP.**
- **Impacto por omisión: Obligaría a implementar la lógica de red y el protocolo HTTP desde cero utilizando los módulos nativos de Node.js, incrementando la complejidad del código.**

5. **Configuración del entorno de desarrollo (Nodemon)**

R//

- **Propósito: Automatizar el reinicio del proceso del servidor en el entorno de desarrollo cada vez que se detecten cambios en el código fuente.**
- **Impacto por omisión: Pérdida de eficiencia en el flujo de trabajo, requiriendo la finalización y el reinicio manual del proceso (node app.js) tras cada modificación.**

6. **Definición del punto de entrada y asignación del puerto**

R//

- **Propósito: Establecer el archivo principal de arranque de la aplicación y configurar el socket de red (puerto) para escuchar conexiones entrantes.**
- **Impacto por omisión: La aplicación carecerá de un hilo de ejecución inicial y el servidor no podrá vincularse a la red para atender peticiones.**

7. **Habilitación del parser global (express.json())**

R//

- **Propósito: Interceptar las peticiones entrantes con encabezado Content-Type: application/json para parsear su contenido.**
- **Impacto por omisión: Incapacidad absoluta del backend para procesar, validar o almacenar la información enviada en el cuerpo de las peticiones.**

8. **Implementación del middleware personalizado**

R//

- **Propósito: Registrar de forma centralizada la actividad del servidor en la consola, facilitando la auditoría y el depurado (*debugging*) en tiempo real.**
- **Impacto por omisión: Pérdida de visibilidad sobre los flujos de tráfico entrantes y dificultad para rastrear errores en el ciclo de vida de las solicitudes.**

9. Diseño y exposición de Endpoints (Rutas GET y POST)

R//

- **Propósito: Definir los puntos de acceso de la API para la consulta de información (GET) y la recepción/persistencia de datos (POST).**
- **Impacto por omisión: La API carecerá de interfaces funcionales, resultando en un servidor activo, pero sin capacidad de respuesta ante el cliente.**

10. Control de versiones y persistencia remota (Git y GitHub)

R//

- **Propósito: Asegurar la trazabilidad del código mediante un historial de cambios e implementar un respaldo seguro en la nube.**
- **Impacto por omisión: Ausencia de copias de seguridad del proyecto y pérdida de la capacidad de colaborar en paralelo o revertir modificaciones erróneas.**

CONCLUSIONES DEL PROYECTO

- **La ejecución de este componente práctico permitió asimilar la arquitectura fundamental de un backend REST utilizando el ecosistema de Node.js y Express.js, logrando estructurar endpoints funcionales bajo estándares técnicos.**
- **Se determinó el rol crítico que desempeñan los middlewares en el flujo de datos; en especial express.json(), cuya ausencia rompe la comunicación asíncrona al invalidar la lectura del formato JSON en el servidor.**
- **La integración de herramientas como Nodemon y clientes de pruebas de API (Thunder Client / Postman) optimizó significativamente el ciclo de desarrollo, validando de forma exitosa tanto las respuestas HTTP del servidor como los registros de auditoría en la terminal.**